

1.AWS Tutorial:

Lesson Objectives:

1.1 Cloud Computing

- Introduction to Cloud Computing
- Real-world examples of Cloud Computing
- Key Benefits of Cloud Computing
 - Cost Effective
 - Scalability
 - Accessibility
 - Reliability

Introduction to Cloud Computing

Cloud computing allows individuals and businesses to access computing resources such as storage, processing power, and applications over the internet, eliminating the need to invest in and maintain physical hardware. Instead of purchasing and managing on-premises servers, users can leverage cloud providers like **Amazon Web Services (AWS)**, **Microsoft Azure**, and **Google Cloud Platform (GCP)** to scale resources as needed.

In simple terms, we can say Cloud computing is the **delivery of computing services** like **networking, servers, storage, databases, and software** over the internet. Instead of owning and maintaining physical infrastructure, users can access these resources on demand from cloud providers.

Real-World Examples of Cloud Computing

Cloud Storage (Google Drive, Dropbox)

Services like Google Drive and Dropbox enable users to store files securely on remote servers, making them accessible from any device with an internet connection.

Streaming Services (Netflix, Spotify)

Platforms like Netflix and Spotify stream movies, shows, and music directly from cloud-based servers, allowing users to access content without the need for downloads.

Email Hosting (Gmail, Outlook)

Email providers such as Gmail and Outlook store emails and related data in the cloud, eliminating the need for users to manage personal email servers while ensuring accessibility and reliability.

Key Benefits of Cloud Computing

Cost-Effective: No need to buy expensive servers; you only pay for what you use.

Scalable: Need more resources? Add them with a few clicks. Need less? Scale down anytime.

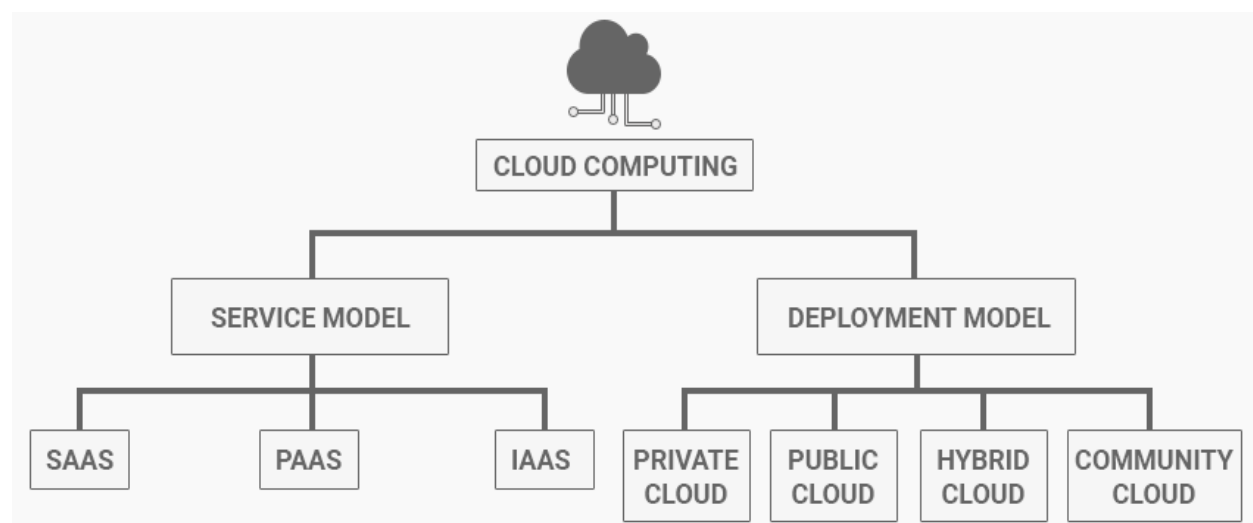
Accessible: Work from anywhere, anytime, as long as you have an internet connection.

Reliable: Top providers ensure 99.9% uptime, meaning your services are rarely down.

1.2 Types of Cloud Computing

- Service Models
 - Infrastructure as a Service
 - Platform as a Service
 - Software as a Service
- Deployment Models
 - Public Cloud
 - Private Cloud
 - Hybrid Cloud
 - Community Cloud

Type of Cloud Computing:



Cloud computing is categorized into three main service models, each offering different levels of control and management:

Infrastructure As a Service (IAAS):

Imagine renting the building blocks of a house—bricks, cement, and tools—but you're responsible for building and maintaining it.

What it is:

IaaS provides raw computing resources like virtual servers, storage, and networks. You manage everything on top of that, like the operating system, apps, and data.

Example in real life:

- Renting a truck to move your belongings—you provide the driver, fuel, and direction.
- Cloud Example: **AWS EC2, Microsoft Azure VMs, Google Compute Engine.**

Use Cases:

- Hosting websites.
- Running custom applications.
- Setting up virtual machines for development.

2. Platform as a Service (PaaS):

Imagine renting a fully equipped kitchen where you just cook your favorite dish. The appliances, gas, and utensils are already set up for you.

What it is:

PaaS offers a ready-to-use platform where developers can build, test, and deploy applications without worrying about the underlying infrastructure.

Example in real life:

- Ordering a pre-made pizza base and adding your favorite toppings.
- Cloud Example: **Google App Engine, AWS Elastic Beanstalk, Microsoft Azure App Services.**

Use Cases:

- Developing apps quickly without setting up servers.

- Running APIs or backend services.
- Prototyping apps for faster deployment.

3. Software as a Service (SaaS):

This is the most convenient model. Imagine ordering a fully prepared meal from a restaurant. You just enjoy the food without worrying about cooking or cleaning.

What it is:

SaaS delivers software applications over the internet, ready to use. You don't need to install or maintain anything.

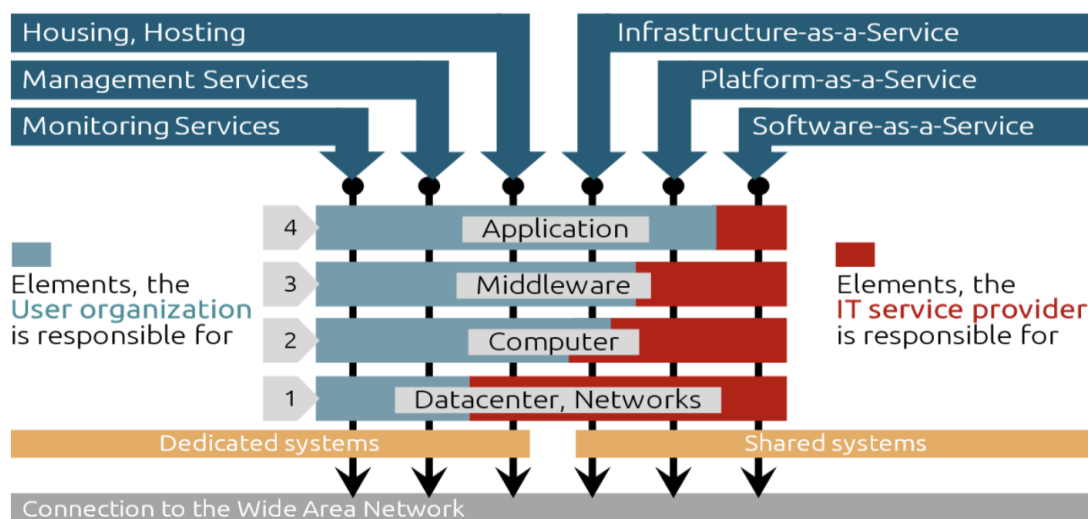
Example in real life:

- Streaming Netflix instead of downloading or buying DVDs.
- Cloud Example: **Gmail, Microsoft Office 365, Zoom, Slack.**

Use Cases:

- Email services.
- Project management tools like Trello.
- CRMs like Salesforce.

The image shows who is responsible for different parts of the IT system in various service models. The blue-green areas (on the left) are managed by the user organization, while the red areas (on the right) are handled by the IT service provider. It also shows which service models usually use dedicated systems, and which use shared systems.



Deployment Model:

Cloud computing offers different ways to deploy its services, depending on your needs. These are called deployment models, and they define where your cloud resources are hosted and who has access to them. Let's break them down in simple terms:

1. Public Cloud:

The public cloud is like a library. It's open to everyone, and you share resources with other users. Big companies like **AWS**, **Azure**, and **Google Cloud** own these public cloud platforms and let you "rent" their servers, storage, and software.

Example:

- Hosting your company's website on **AWS EC2**. You don't need to own the servers; AWS manages everything.

Pros:

- **Affordable**: You only pay for what you use.
- **Scalable**: Add more resources anytime.
- **Accessible**: Access from anywhere with the internet.

Cons:

- Shared environment may have **security concerns** for sensitive data.

2. Private Cloud:

The private cloud is like owning your personal library. Only your company can access it, and the resources are dedicated entirely to you. It's hosted either on-site (your own servers) or by a third party exclusively for you.

Example:

- A bank managing its sensitive customer data in its private cloud for enhanced security.

Pros:

- **Secure**: Dedicated infrastructure reduces data risks.
- **Customizable**: Tailor it to your specific needs.

Cons:

- **Expensive**: High setup and maintenance costs.
- **Limited scalability**: Adding resources may take time.

3. Hybrid Cloud:

The hybrid cloud is like a mix of a library and personal collection. You keep some resources private while using public resources for the rest. It combines the best of both worlds.

Example:

- An e-commerce company storing sensitive customer data (private cloud) but hosting the website on a public cloud to handle high traffic.

Pros:

- **Flexible:** Move workloads between private and public clouds.
- **Cost-Effective:** Use public cloud for non-sensitive tasks.
- **Scalable:** Handle fluctuating workloads efficiently.

Cons:

- **Complex:** Managing two environments can be tricky.
- **Integration Issues:** Requires robust networking.

4. Community Cloud:

The community cloud is like a private library shared by a group of people with common interests. Organizations with similar needs share the same cloud infrastructure.

Example:

- Universities collaborating on research projects using a shared cloud platform.

Pros:

- **Cost Sharing:** Expenses are divided among participants.
- **Collaboration:** Ideal for joint projects or industries.

Cons:

- **Limited Availability:** Not as common as other models.
- **Shared Security Risks:** All users must follow strict security practices.

Which Model to Choose?

- **Public Cloud:** Best for startups, small businesses, and anyone looking for affordable, scalable solutions.

- **Private Cloud:** Ideal for industries like banking, healthcare, or government that require strict security.
- **Hybrid Cloud:** Perfect for companies needing both security and flexibility, such as large enterprises.
- **Community Cloud:** Suited for collaborative projects in industries like education or research.

Simplified Analogy

- **Public Cloud:** Renting a seat in a co-working space.
- **Private Cloud:** Owning your office.
- **Hybrid Cloud:** Splitting your work between your office and a co-working space.
- **Community Cloud:** Sharing an office with like-minded professionals.

Model → Property ↓	On-premise (in-house operation)	Private Cloud	Virtual Private Cloud	Public Cloud
User	operator/provider	associated with operator/provider	associated with operator/provider	unbound or loosely connected
	trustworthy	trustworthy	trustworthy	unknown
Users share cloud instance with	nobody	nobody	several	many
Access (network)	local network, campus network, etc.	Wide area network with restricted access	Wide area network with restricted access	public Internet
Place of production	user organization	IT service provider (defined location)	IT service provider (defined location)	IT service provider (location often not defined)

Conclusion:

Cloud deployment models offer flexibility to businesses, enabling them to choose the right setup based on their needs, budget, and security requirements. Whether it's the cost-effectiveness and scalability of the public cloud, the security and control of the private cloud, or the balance of flexibility in the hybrid cloud, there's a solution for everyone.

- For organizations collaborating on shared goals, the **community cloud** provides a unique opportunity to pool resources efficiently.

By understanding these deployment models, businesses can make informed decisions to optimize costs, enhance performance, and ensure data security, paving the way for seamless growth in a digital-first world.

Cloud computing revolutionizes IT infrastructure by providing flexible, cost-efficient, and scalable solutions. In this course, we'll dive deeper into AWS services and how you can leverage them effectively.

1.3 Lesson Objectives:

- Why AWS?
- Introduction to Amazon Web Services
- History of AWS
- Overview of AWS Global Infrastructure
 - Regions and Availability Zones
 - Global Network Infrastructure

Why AWS?

Now that we have a solid understanding of what cloud computing is and how it transforms the way businesses manage IT resources, it's time to explore one of the most powerful platforms enabling this transformation--**Amazon Web Services (AWS)**. As a pioneer and global leader in cloud computing, AWS provides a wide range of cloud-based services that allow organizations to build, deploy, and manage applications and infrastructure more efficiently and securely. In this course, we will dive deep into AWS, exploring its core services, architecture, and real-world use cases to help you leverage the full potential of cloud computing.

1. Market Leadership & Trust

- AWS has been a pioneer in the cloud computing space since 2006.
- It dominates the cloud market with the largest share, trusted by major companies like Netflix, NASA, and Airbnb.

2. Global Reach & Availability

- AWS operates in 30+ geographic Regions with 100+ Availability Zones (AZs) around the world.
- This ensures low latency, fault tolerance, and high availability for global applications.

Example: If your app needs users from the US, Europe, and Asia, you can deploy instances in different AWS Regions close to those users for faster access.

3. Flexible and Transparent Pricing

- AWS follows a pay-as-you-go and pay-less-when-you-reserve model.
- For beginners, AWS offers a Free Tier that includes services like EC2, S3, Lambda, and RDS for free up to certain limits.

Example: You can host a static website using S3 under the Free Tier at zero cost!

4. Security and Compliance

- AWS provides end-to-end encryption, IAM roles, MFA, and network isolation through VPC.
- It complies with global security standards like ISO 27001, SOC 1/2/3, PCI-DSS, and HIPAA.

Good to Know: AWS has a shared responsibility model—AWS secures the cloud infrastructure; you secure your data inside it.

5. Scalability and Elasticity

- AWS services like EC2 Auto Scaling, Elastic Load Balancing, and Lambda let you scale your resources automatically based on demand.
- This ensures your app performs well even during peak times without manual intervention.

Example: An e-commerce app during Black Friday can auto-scale using EC2 instances based on traffic spikes.

6. Breadth of Services

- AWS offers 200+ fully featured services across compute, storage, databases, analytics, machine learning, IoT, security, and DevOps.
- These services are deeply integrated and work well together, allowing seamless architecture designs

Introduction to Amazon Web Services:

As a leading cloud computing platform, Amazon Web Services (AWS) provides a vast array of web services spanning multiple domains, enabling businesses to optimize performance and scale efficiently. AWS continuously evolves to meet the demands of modern IT, offering solutions across compute, storage, networking, databases, and more. Its extensive service offerings cater to a diverse range of industries, empowering organizations to enhance their operations and drive innovation.

In this course material, we will cover the fundamentals of AWS, exploring its key services and its significance in shaping the future of IT infrastructure.

History of AWS:

Amazon Web Services (AWS) started as an internal infrastructure project within Amazon.com in the early 2000s. The company needed a more scalable and reliable IT infrastructure to handle its growing e-commerce business. During this process, Amazon realized that other companies could benefit from a similar cloud-based infrastructure.

In 2003, Amazon executives, including Chris Pinkham and Benjamin Black, proposed the idea of a standardized, scalable computing infrastructure that businesses could use on a pay-as-you-go model. This concept laid the foundation for AWS as a cloud computing service provider.

AWS was officially launched on March 14, 2006, with the introduction of three core cloud services:

- **Amazon S3 (Simple Storage Service):** A scalable storage service.
- **Amazon SQS (Simple Queue Service):** A messaging service for distributed applications.
- **Amazon EC2 (Elastic Compute Cloud):** Virtual servers that allow users to rent computing power.

These services allowed businesses to store data, process workloads, and build applications without investing in physical hardware. AWS's pay-as-you-go pricing model revolutionized the IT industry, making cloud computing accessible to companies of all sizes. AWS continues to redefine IT infrastructure, enabling businesses to innovate faster and scale efficiently in a cloud-driven world.

Overview of AWS Global Infrastructure:

Before exploring AWS services, it's essential to understand the core infrastructure that powers AWS globally. AWS is designed to offer high availability, scalability, and reliability through its Regions, Availability Zones (AZs), and Global Network Infrastructure.



Regions: AWS provide the services with respective division of regions. The regions are divided based on geographical areas/locations and will establish data centers. Based on need and traffic of users, the scale of data centers is depended on to facilitate users with low-latencies of services.

Example:

- us-east-1 (N. Virginia)
- eu-central-1 (Frankfurt)
- ap-south-1 (Mumbai)

Availability Zones (AZ): To prevent the Data centers for the Natural Calamities or any other disasters. The Datacenters are established as sub sections with isolated locations to enhance fault tolerance and disaster recovery management.

Example:

- us-east-1 has 6 AZs (e.g., us-east-1a, us-east-1b, etc.)
- eu-central-1 has 3 AZs

Global Network Infrastructure: AWS ensures the reliability and scalability of services through setting up its own AWS Network Infrastructure globally. It helps in better management of data transmissions for optimized performance and security reliance. To learn more about AWS Global Infrastructure follow [Explore AWS Global Infrastructure](#) .

Lesson Objectives:

1.4 AWS Core Services

- AWS Core Services
 - Compute Services
 - Storage Services
 - Networking Services
 - Developer Tools
- Hands-on Exercise
 - Launch an EC2 Instance
 - Connect to an Instance
 - Hosting an Instance

AWS Core Services:

In the rapid revolution of Cloud Computing, AWS facilitates with wide variety of services respect to the fields and needs. The following are the top AWS services that are in wide usage:

1. Compute Services

- **Amazon EC2 (Elastic Compute Cloud):** EC2 provides resizable compute capacity in the cloud. It allows users to run virtual servers (instances) that can be customized with different configurations based on needs. EC2 offers various instance types optimized for different workloads, including compute, memory, and storage-intensive tasks. Users can start, stop, and scale instances based on their requirements.
- **AWS Lambda:** A serverless compute service that lets you run code without provisioning or managing servers. You upload your code (functions), and Lambda handles the scaling and infrastructure management automatically. It

charges based on execution time rather than uptime, making it cost-efficient for event-driven workloads.

- **Amazon Elastic Beanstalk:** A Platform-as-a-Service (PaaS) offering that simplifies deploying, managing, and scaling applications. You upload your code, and Elastic Beanstalk automatically handles the deployment, from provisioning resources to scaling and monitoring the application.
- **Amazon LightSail:** A simplified cloud platform designed for small to medium-sized applications, providing pre-configured virtual private servers with managed databases, networking, and storage features. It offers a simpler alternative to EC2 for those who need an easy-to-use environment.
- **Amazon ECS (Elastic Container Service):** A fully managed container orchestration service that supports Docker containers. ECS allows users to deploy and manage containerized applications with ease, using AWS infrastructure for scaling and security.
- **Amazon EKS (Elastic Kubernetes Service):** A fully managed service for running Kubernetes clusters on AWS. EKS automates the setup, scaling, and operations of Kubernetes, making it easier for users to run containerized applications.

2. Storage Services

- **Amazon S3 (Simple Storage Service):** A scalable object storage service that allows you to store and retrieve large amounts of data. S3 provides high durability, availability, and security. It's ideal for storing backups, static website assets, big data analytics, and more.
- **Amazon EBS (Elastic Block Store):** A block storage service designed to be used with EC2 instances. EBS provides persistent storage with low-latency access. It's suitable for use cases that require a database, file system, or any application that needs block-level storage.
- **Amazon EFS (Elastic File System):** A scalable file storage service that allows multiple EC2 instances to access a shared file system. It provides a simple, scalable solution for file storage, making it suitable for applications that require shared access to files.
- **Amazon Glacier:** A low-cost, long-term archival storage service for data that is infrequently accessed. It is optimized for backups, disaster recovery, and data archiving.

3. Networking Services

- **Amazon VPC (Virtual Private Cloud):** A logically isolated network within AWS that allows you to define and control your cloud network, including IP addresses, subnets, route tables, and gateways. VPC lets you securely connect your AWS resources.
- **Amazon Route 53:** A scalable domain name system (DNS) web service that routes end-user requests to applications, websites, and resources in AWS. It can be used for domain registration, DNS routing, and health checking.
- **AWS Direct Connect:** A dedicated network connection between your on-premises data center and AWS. It provides a more reliable, consistent network experience than typical internet-based connections.
- **Amazon CloudFront:** A content delivery network (CDN) that caches copies of your content at edge locations to provide low-latency delivery to users globally. CloudFront speeds up the delivery of your static and dynamic content.

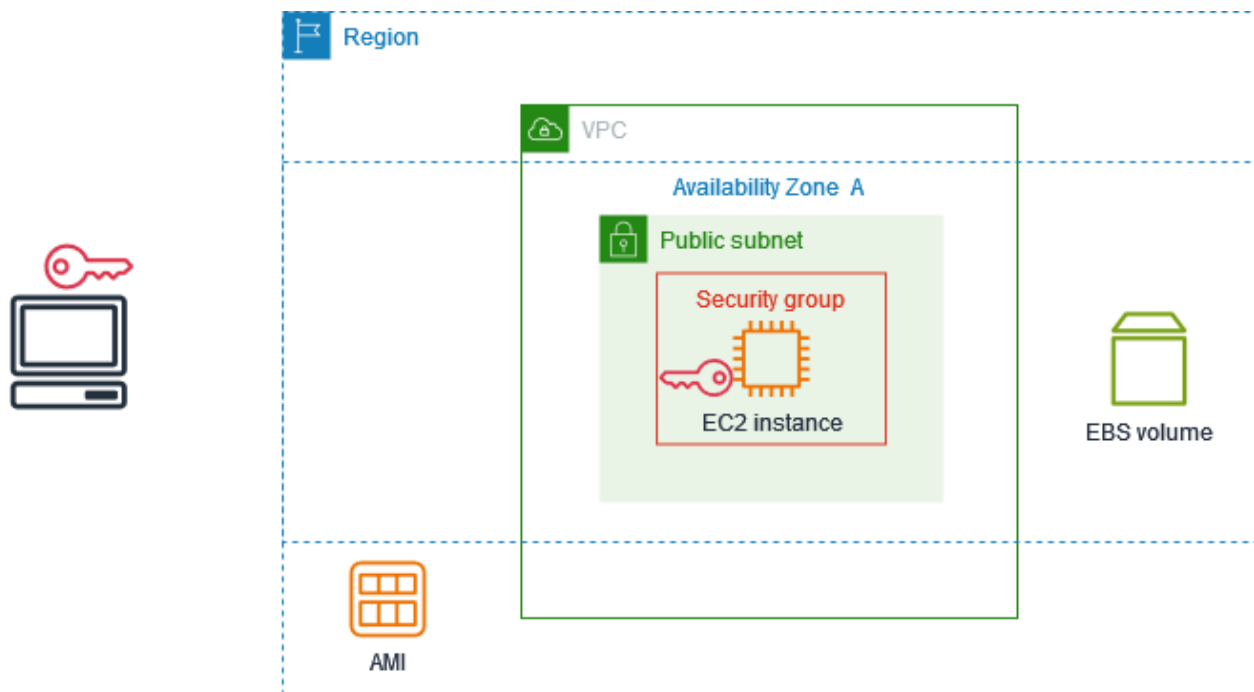
4. Developer Tools

- **AWS CodeCommit:** A fully managed source control service that allows you to host secure Git repositories for your code.
- **AWS CodeBuild:** A fully managed build service that compiles source code, runs tests, and produces ready-to-deploy software packages.
- **AWS CodeDeploy:** A service that automates the deployment of applications to various compute services, such as EC2, Lambda, and on-premises servers.
- **AWS CodePipeline:** A fully managed continuous integration and continuous delivery (CI/CD) service that automates the building, testing, and deployment of code.

AWS provides an extensive array of services that cater to a wide range of cloud computing needs, from compute and storage to security, analytics, and machine learning. These services are designed to work together to enable scalable, secure, and cost-effective solutions. As you explore AWS, you can combine these services to meet your specific use cases and optimize your infrastructure based on requirements such as performance, cost, and complexity. To understand and explore more of these services go through the link [AWS Service List](#).

Before stepping into the Hands-on let us understand the overview of the key components that we will see in this tutorial:

- **An image:** A template that contains the software to run on your instance, such as the operating system.
- **A key pair:** A set of security credentials that you use to prove your identity when connecting to your instance. The public key is on your instance and the private key is on your computer.
- **A network:** A virtual private cloud (VPC) is a virtual network dedicated to your AWS account. To help you get started quickly, your account comes with a default VPC in each AWS Region, and each default VPC has a default subnet in each Availability Zone.
- **A security group:** Acts as a virtual firewall to control inbound and outbound traffic.
- **An EBS volume:** We require a root volume for the image. You can optionally add data volumes.



Hands-on Exercises:

Now that you've learned the core AWS services (Compute, Storage, Databases, Networking, Monitoring), let's start using them with real AWS resources. These hands-on labs will guide you from creating an AWS account to deploying simple applications.

Exercise-1: Create an AWS Account

Step by step guide:

1. Go to the AWS Website

- Open your browser and go to: <https://aws.amazon.com>
- Click on “**Create an AWS Account**” (usually top right).

2. Enter Account Information

1. Enter your **email address**.
2. Choose a **password** (must be strong).
3. Choose an **AWS account name** (e.g., my-cloud-lab, devops-student123).
4. Click **Continue**.

3. Choose Account Type

You’ll be asked to choose between:

- **Personal** – for learning or personal projects (most people choose this)
- **Professional** – for company use
- Enter all the required details and click continue.

4. Payment Information

- Enter your credit or debit card details (AWS won’t charge anything immediately).
- Enter billing address.
- Click Verify and Add.

Note: AWS may charge a temporary \$1 authorization just to verify your card. It will disappear soon.

5. Phone Number Verification

- Choose your country code, enter your mobile number.
- Choose SMS or voice call.
- Enter the verification code sent to you.

Click Continue once verified.

6. Select a Support Plan

You’ll see three options:

- Basic (Free) – choose this one!
- Developer
- Business

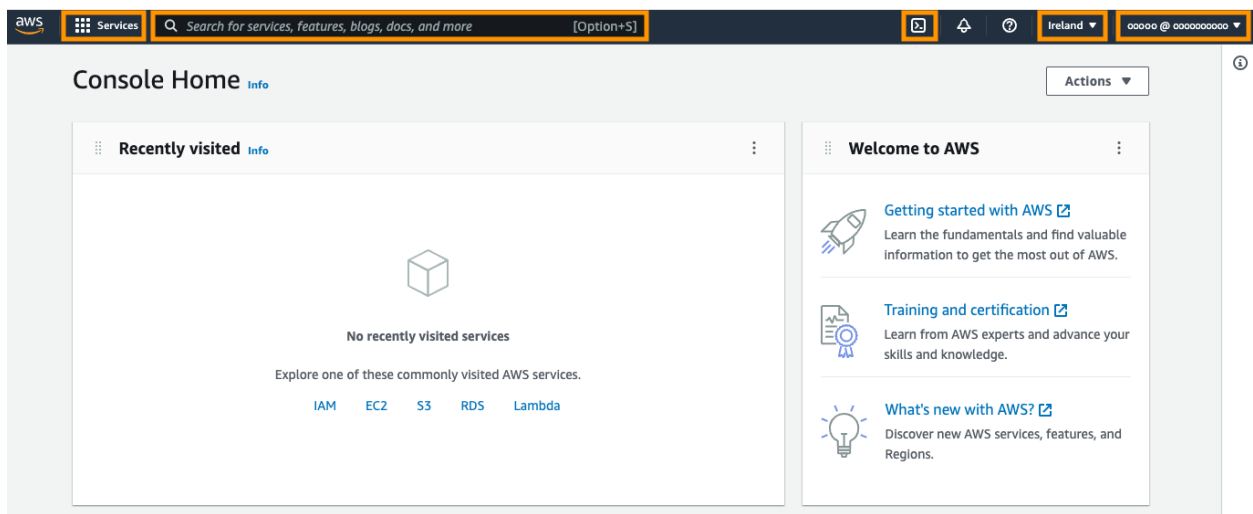
Click Free Plan.

Free Tier explained:

7. Finalize and Sign In

Click Sign in to the Console, then:

- Choose Root user
- Enter your email and password
- You'll land on the AWS Management Console



Exercise-2:

Launching and Connecting to an EC2 instance:

1. Sign In to AWS Console

- Go to <https://aws.amazon.com>
- Click Sign In > use your root user credentials
- Navigate to EC2 by typing it in the search bar

2. Launch EC2 Instance

1. Click "Launch Instance"
2. Fill in these details:
 - Name: Anything (My Webserver)
 - AMI: Choose Amazon Linux 2 or Ubuntu (free tier eligible)

- Instance Type: Choose t2.micro (free tier)
- Key Pair:
 - Create a new key pair or use existing
 - Download the .pem file (important for SSH access)

Warning

If you choose **Proceed without a key pair (Not recommended)**, you won't be able to connect to your instance using the methods described in this tutorial.

- Network Settings:
 - Allow HTTP, HTTPS, and SSH traffic (choose "Allow" from internet)
 - Storage: Default 8GB (can keep as is)
3. Click Launch Instance

Click on the link below for more details on creating and launching an instance.

[Launching an EC2 Instance](#)

4.Connect to EC2 Instance

After the instance state shows “Running”:

Locate Your Instance's Public IP:

- In the EC2 Dashboard, find your running instance and note its Public IPv4 address.

Connect via SSH:

- Open your terminal or command prompt.
- Navigate to the directory containing your key pair file.
- Modify the key's permissions for security and connect to your instance using SSH. [AWS EC2](#)

```
bash
```

```
chmod 400 your-key.pem
```

```
ssh -i "your-key.pem" ec2-user@<public-ip>
```

Exercise-3: Hosting a Website on EC2

1.Install Web Server (Apache)

Once connected via SSH, run the following commands:

```
bash

# For Amazon Linux 2
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd

# For Ubuntu
sudo apt update
sudo apt install apache2 -y
sudo systemctl start apache2
sudo systemctl enable apache2
```

2.Add Web Content

Replace the default web page with your content:

```
bash

cd /var/www/html
sudo echo "<h1>Hello from EC2!</h1>" > index.html
```

You can upload your own HTML files using **SCP**:

```
scp -i your-key.pem index.html ec2-user@<public-ip>:/var/www/html/
```

3.View the Website

- Open a browser
- Enter your EC2 Public IP
- You should see: Hello from EC2! confirming that it's successfully hosted on your EC2 instance.

Ref Link:

[Connect to an EC2 Instance](#)

Video Tutorials:

[To Launch and hosting a website](#) (Launch EC2, install Apache, host HTML site)

This guide provides a detailed overview of hosting a website on AWS EC2, complete with step-by-step instructions and visual aids. By following these steps, you can effectively set up and host a website on an AWS EC2 instance.

Exercise-4:

Terminating an EC2 Instance

After you've finished with the instance that you created for this tutorial, you should clean up by terminating the instance.

To terminate your instance

1. In the navigation pane, choose Instances. In the list of instances, select the instance.
2. Choose Instance state, Terminate instance.
3. Choose Terminate when prompted for confirmation.

Important

Terminating an instance effectively deletes it; you can't reconnect to an instance after you've terminated it.

Amazon EC2 shuts down and terminates your instance. After your instance is terminated, it remains visible on the console for a short while, and then the entry is automatically deleted. You cannot remove the terminated instance from the console display yourself.

References:

Complete AWS Video tutorials:

<https://youtu.be/n6RWhajimZg?si=JokwZdfWNZ2cBB6u>

GitHub Repo: <https://github.com/iam-veeramalla/aws-devops-zero-to-hero>

For Exercises: https://www.w3schools.com/aws/aws_exercises.php

Quiz: https://www.w3schools.com/aws/aws_quiz.php